Partner Integration BFF

Overview

Thank you for your interest in joining our team! Before our technical interview, we would like you to complete a short coding exercise.

The Scenario: Our platform is transitioning to an asset-light, partner-driven model. We need to build a new Backend-for-Frontend (BFF) microservice in **.NET 8**. This service will receive incoming transaction data from third-party partners, validate it, enrich it via an external API, and reliably queue it for our legacy systems to process.

The Requirements

Please create a **.NET 8 Web API** project that accomplishes the following:

1. The Endpoint `POST /api/v1/partner/transactions`

- Accepts a JSON payload representing a partner transaction.

  JSON

  ```
  {
    "partnerId": "P-1001",
    "transactionReference": "TXN-99823",
    "amount": 250.00,
    "currency": "USD",
    "timestamp": "2024-05-10T14:30:00Z"
  }
  ```

- Validates the payload: Amount must be > 0, currency must be valid, all fields required

2. External Service Integration

- Before accepting the transaction, the service must verify the `partnerId` against a mock "Partner Verification API".
- **Requirement:**
  - Implement a dummy API endpoint (in the same solution/project) that randomly throw a `TimeoutException` 30% of the time, and return valid response 70% of the time
  - Call that API to validate the `partnerId`
  - Implement a resilience strategy to handle retries and failures gracefully without crashing the incoming request.

3. Asynchronous Messaging

- If the payload is valid and the partner is verified, the transaction should be sent to a message broker.
- **Requirement:**
    - Spin up a message queue locally.
    - Create the interface and concrete implementation to send the message to queue

4. Quality & Testing

- Write unit tests (using xUnit or NUnit) covering your validation logic and the resilience/retry mechanism.
- High code coverage is important to us.

Bonus Points (Optional, but highly regarded)

- Containerize the application and include a `docker-compose.yml` that spins up both your API and the local message queue.
- Implement a simple Global Exception Handler to format error responses consistently.
- Demonstrate how you would secure this endpoint.

Submission Guidelines

- Please provide your solution as a link to a public GitHub/GitLab repository.
- Include a short `README.md` explaining your architectural choices, how to run the project, and how to run the tests.
- *Note: We expect this to take roughly 2-3 hours. We are evaluating your architecture, clean code principles, and testing strategies, not just a working script.*